

Mini Militia-style Mobile Shooter — Architecture & Development Plan

0. Verdict first

All five requirements are feasible for a solo dev. None of them are research problems. The original Mini Militia shipped this exact feature set in 2011 on far worse hardware.

Engine: Unity 6.3 LTS. Not Godot, not Unreal. Reason below — it comes down to one requirement.

The one architecture: a tick-based, authoritative listen-server (one player's phone is both server and client), with a **swappable transport**. LAN and online share the *same* networking layer and the *same* code. Only connection establishment differs.

Voice chat is online-only. LAN/hotspot matches ship with no voice — people within hotspot range are almost always in the same room and can simply talk. This removes voice from the MVP entirely, cuts ~2 weeks and the **RECORD_AUDIO** permission off your first release, and *unlocks cloud voice services* for online mode, which are the better tool once you can assume an internet connection.

Should you use Unity, or something else?

Engine	Verdict
Unity 6.3 LTS	✅ Use this.
Godot 4.6	Genuinely good for 2D, free forever, smaller APK, multi-instance debugging built in. But: its high-level multiplayer API has no built-in client prediction — you would hand-roll rollback and reconciliation, which is the single hardest part of this project.
Unreal 5	Overkill. ~80 MB+ base APK, heavy editor, 2D workflow is poor, mobile iteration is slow.
Cocos / Defold	Fine 2D engines, but the multiplayer networking ecosystems are thin.
Web (JS) + Capacitor	LAN UDP discovery is essentially impossible from a WebView. Disqualified.

The deciding factor is **free, battle-tested client-side prediction**. FishNet gives it to you in Unity for nothing; in Godot you would write rollback and reconciliation from scratch, and that is where solo multiplayer projects most often die. Unity also brings a far larger Android troubleshooting corpus, mature mobile input assets, and a deep Asset Store for the art and UI you will need in Phase 5.

Honest note: making voice online-only narrows Unity's lead. The old clincher was that Dissonance solved offline-LAN voice off the shelf and Godot had no equivalent. With that requirement gone, **Godot 4.6 is a genuinely closer call** than it used to be — it's still the wrong pick here, but on prediction tooling and ecosystem depth rather than on voice.

Unity's licensing is no longer a risk factor: the Runtime Fee was permanently cancelled, Personal is free up to **\$200k** revenue/funding, and the splash screen is now removable on Personal.

Use Unity 6.3 LTS (supported to Dec 2027), **2D URP**, **IL2CPP + ARM64**.

1. Feasibility analysis

Requirement	Feasible?	The honest caveat
LAN / hotspot multiplayer	✔ Yes	Easiest part technically. The hard part is <i>discovery UX</i> on Android — broadcast packets get dropped by some devices/power-save modes. You must build manual-IP and QR fallbacks. Non-negotiable.
Internet multiplayer	✔ Yes	Solved problem if you use a relay. Do not attempt DIY NAT punchthrough. Add it <i>after</i> the LAN version ships.
Real-time voice chat	✔ Yes — online mode only	Scoped to online matches, so a cloud voice service (EOS RTC / Vivox) is available and is the right choice: better echo cancellation, and it keeps voice traffic off the host's connection entirely. LAN mode ships with no voice by design.
2–6 players	✔ Trivially	Your entire state is ~130 bytes/packet. This is a rounding error on bandwidth. Voice is 20× heavier than gameplay.
Android first	✔ Yes	Real risks are thermal throttling, app backgrounding killing connections, and device fragmentation — not the networking.

What actually makes this project hard isn't any single item. It's that you're building a *real-time simulation*, a *lobby/session state machine*, a *discovery layer* and *mobile game feel* — four disciplines — alone, with a *voice pipeline* as a fifth once online mode arrives. The plan below is optimized to de-risk them in the right order, and deliberately defers voice past your first release.

2. Game architecture

2.1 The layering (this is the core idea)

PRESENTATION networked	UI, HUD, VFX, SFX, camera, animation	never
SIMULATION FishNet refs	PlayerMotor.Simulate(input, dt)	plain C#, NO
	WeaponSim, DamageResolver, MatchRules	
deterministic, re-runnable		
REPLICATION	NetworkObject, SyncVars, RPCs, prediction/reconciliation, interest mgmt	FishNet

TRANSPORT	[SWAPPABLE] LAN → Tugboat (UDP/LiteNetLib) Online → Relay transport
DISCOVERY	[SWAPPABLE] LAN → UDP broadcast beacon Online → Lobby service + join code

The rule that saves you months: the Simulation layer must not reference FishNet at all. `PlayerMotor` is a plain C# class with `Simulate(PlayerInput input, float dt)` returning a `PlayerState` struct. This gives you three things for free:

- 1. Reconciliation becomes cheap — you just re-run `Simulate()` 8 times in one frame.
- 2. You can unit-test movement without spinning up a network.
- 3. If you ever need to swap networking libraries, you rewrite one folder, not the game.

2.2 Unity client architecture

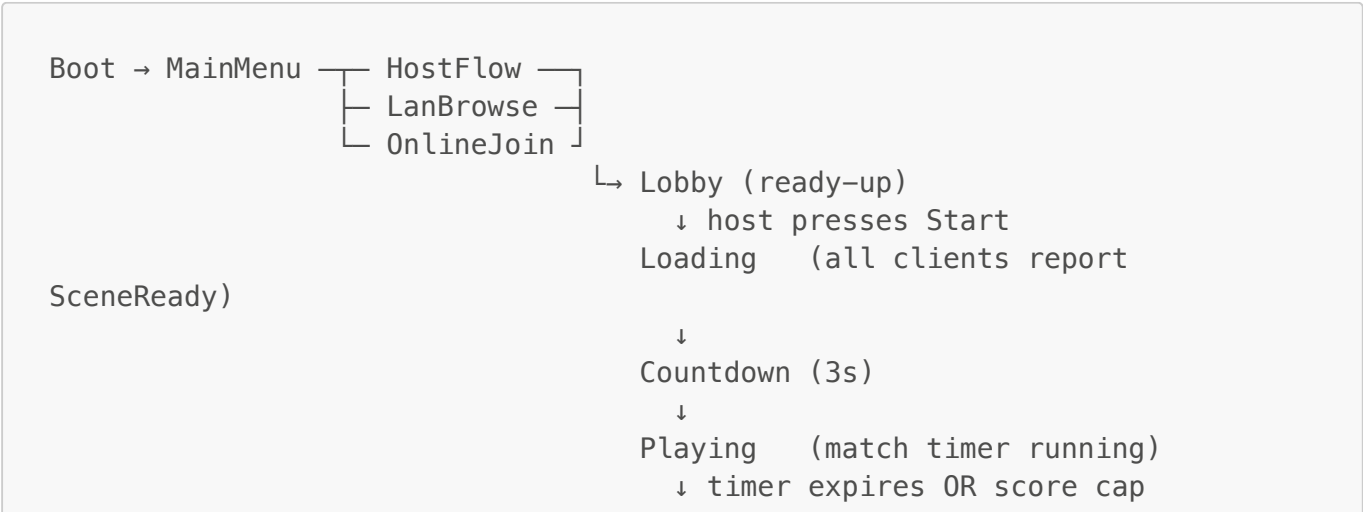
Single process, three roles it can occupy:

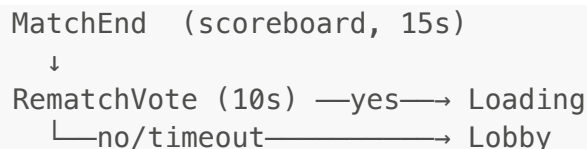
Role	Runs simulation?	Renders?
Host (server + client)	✔ authoritative	✔
Client	predicts local player only	✔
Dedicated server (Phase 7, same build, <code>-batchmode -nographics</code>)	✔ authoritative	✘

A `NetworkBootstrap` singleton in scene `00_Boot` persists across everything and owns the FishNet `NetworkManager`. Scenes below it get loaded/unloaded; the network session never does.

Scene flow: `00_Boot` → `10_MainMenu` → `20_Lobby` → `30_Map_Foundry` → back to `20_Lobby` (for rematch).

2.3 Match / session state machine





Critical: Lobby → Playing → MatchEnd → Lobby never tears down the network session. Rematch is a state transition, not a reconnect. This is exactly what made Mini Militia feel good — you're back in a new match in ~4 seconds. If you rebuild the session each round, the game feels dead.

2.4 Player state synchronization

Two separate channels, deliberately:

Client → Host (input, 30 Hz, unreliable):

```

struct PlayerInput {           // ~10 bytes
    uint  tick;                // 4
    sbyte moveX;               // 1  (-127..127 quantized stick)
    ushort aimAngle;           // 2  (0..65535 → 0..360°)
    byte  buttons;             // 1  (jetpack|fire|grenade|reload|swap)
}

```

Every packet carries the **last 3 inputs**, not just the newest. A dropped packet then costs nothing — the host already has that tick. This is a 3-line change that eliminates most perceived packet-loss stutter. Best cost/benefit ratio in the whole netcode.

Host → Client (snapshot, 20 Hz, unreliable):

```

struct PlayerSnapshot {        // ~15 bytes/player
    byte  slotId;
    short posX, posY;          // quantized to 1/64 unit
    short velX, velY;
    ushort aimAngle;
    byte  health;
    byte  flags;               // grounded|jetpacking|dead|invuln|firing
}

```

6 players ≈ 90 B payload + ~40 B headers ≈ **130 B @ 20 Hz = 2.6 KB/s down, ~1 KB/s up**. Utterly trivial. You will never be gameplay-bandwidth-constrained at this scale. Voice is several times heavier, which is why §6 deliberately routes it through a voice provider rather than through the host phone.

Never sync: particles, muzzle flashes, sounds, camera, UI, animation state, ragdolls. Those are *derived* from replicated events on each client independently.

2.5 Physics synchronization

Do not use Rigidbody2D dynamics for players. Unity's 2D physics is not deterministic across devices or across re-simulation, and it will fight your reconciliation forever.

Use a **kinematic character controller**: your own gravity, velocity integration, and **Physics2D.BoxCast** against a static Tilemap collider. Mini Militia movement is simple enough that this is ~250 lines:

```
velocity.y -= gravity * dt
if (jetpackHeld && fuel > 0) { velocity.y += thrust * dt; fuel -= drain * dt; }
velocity.x = moveX * speed
→ BoxCast horizontally, resolve
→ BoxCast vertically, resolve
→ clamp to map bounds
```

This is deterministic to float tolerance, cheap enough to re-run 10x per frame during reconciliation, and gives you total control over game feel.

Rigidbodys are fine for throwables (grenades) and debris — but grenades should be **host-simulated and snapshot-replicated**, not predicted. There are at most 3 in the air; just sync their transforms at 20 Hz.

2.6 Lobby, rooms, joining

One **MatchRoster** object, host-owned, replicated to all:

```
PlayerSlot {
    byte    slotId;           // 0..5, stable for the session
    Guid    playerGuid;       // persistent, from PlayerPrefs – the real
identity
    int     connectionId;     // transport-level, CHANGES on reconnect
    string  displayName;      // ≤16 chars, sanitized
    byte    colorIndex;
    bool    isReady;
    int     kills, deaths;
    SlotState state;          // Empty | Connected | Disconnected | Reserved
    float   disconnectedAt;
}
```

Key design point: everything keys off **playerGuid**, never **connectionId**. Connection IDs are transport-level and change on every reconnect. This one decision is what makes reconnection work at all.

2.7 Reconnection handling

1. On first launch, generate **Guid.NewGuid()** → **PlayerPrefs["player_guid"]**. This is the player's identity forever.
2. On disconnect, host sets **slot.state = Disconnected**, records the time, and **reserves the slot for 45 seconds**. Score and kills are preserved. Avatar is despawned.
3. Client auto-retries the same endpoint: 5 attempts over 20 seconds with backoff (1s, 2s, 4s, 6s, 7s).

4. On reconnect, the client's handshake sends `playerGuid`. Host matches it to the reserved slot → restores score, re-spawns the player, sends a full state snapshot.
5. After 45 s, slot frees up for new joiners.

Mobile-specific trap: when a player gets a phone call or backgrounds the app, Unity pauses and the connection times out. Hook `OnApplicationPause(true)` and fire a `PlayerBackgrounded` RPC first so the host marks them as suspended rather than kicked. Set transport timeout generously (~10 s) — desktop defaults of 5 s are too aggressive for phones.

2.8 Disconnect handling

Case	Behaviour
Clean leave (pressed Quit)	RPC <code>LeaveMatch</code> → slot freed immediately, no 45 s hold
Timeout	Slot held 45 s, avatar despawned, name greyed in scoreboard
Host leaves	Match ends for everyone (see below)
Last player leaves	Host auto-returns to main menu

Always warn the host: *"You're hosting — leaving ends the match for everyone."* Cheap to add, prevents a lot of frustration.

2.9 Host migration — recommendation: **don't build it**

Host migration is a 3–4 week project (state snapshot replication, deterministic election, endpoint rediscovery, re-handshake for every client) and on LAN it usually **cannot work anyway** — the host is normally also the hotspot owner, so when they leave the Wi-Fi network itself disappears. There is nothing to migrate to.

Build this instead (2 days, ~90% of the value):

- Host leaves → all clients see "Host left the game" and are dropped straight back into the LAN room browser with a scan already running.
- Player names, colors and scores are cached locally.
- The next person taps "Create Room" → back in a match in ~5 seconds.

Players understand "host left, someone re-host" instantly. They do not understand a half-second freeze followed by a subtly broken game state. Revisit true migration only after online mode ships, and only for online.

2.10 LAN vs online: same layer or different?

Same networking layer. Emphatically.

Everything from the Replication layer upward is byte-identical between LAN and online. What differs:

	LAN	Online
Discovery	UDP broadcast beacon	Lobby service + join code

	LAN	Online
Transport	Tugboat — direct UDP to 192.168.x.y:7770	Relay transport
NAT	N/A	Handled by relay
Auth	None	Anonymous device auth
Interpolation buffer	~50 ms	~100 ms
Everything else	identical	identical

Concretely, this means two interfaces:

```
interface ISessionDiscovery {           // LanDiscovery |
OnlineLobbyDiscovery
    Task<IReadOnlyList<RoomInfo>> FindRooms(CancellationToken ct);
    Task<RoomHandle> CreateRoom(RoomConfig cfg);
}
interface ITransportProvider {         // LanTransport | RelayTransport
    Task<ServerEndpoint> StartHost(RoomHandle h);
    Task ConnectClient(RoomInfo room);
}
```

Swap the implementation at the main menu based on which button was pressed. FishNet's **Multipass** transport can even run both simultaneously, so a single host could accept LAN *and* internet players — a nice trick for later.

3. Networking technology comparison

	FishNet	Unity NGO 2.x	Photon Fusion 2	Mirror	Custom UDP
LAN / fully offline	✅ Direct IP, zero cloud	✅ Direct IP	❌ Requires Photon Cloud	✅ Direct IP	✅
Internet	✅ via relay transport (EOS / Edgegap / Unity Relay)	✅ via Unity Relay	✅ Best-in-class	✅ via 3rd-party relay	You build it
Scalability (2–6p)	Far beyond need	Far beyond need	Far beyond need	Far beyond need	—
Latency / perf	Excellent	Good	Excellent	Good	Depends
Client prediction	✅ Built in, free	⚠️ Anticipation API, immature	✅ Built in, excellent	❌ Not built in	DIY (months)

	FishNet	Unity NGO 2.x	Photon Fusion 2	Mirror	Custom UDP
Lag compensation	✓ ColliderRollback (Pro)	✗	✓	✗	DIY
Ease of development	Moderate (docs thinner)	Easiest docs	Easy <i>if cloud-only</i>	Easy, huge community	Very hard
Server requirements	None (host mode)	None (host mode)	Photon Cloud mandatory	None	Yours
Cost	Free ; Pro ~\$10 one-time	Free + metered Relay	Free ≤100 CCU, then \$125/mo @500, \$250 @1000	Free	Infra + your time
Voice integration (online only)	EOS RTC, Vivox, or Dissonance (community pkg)	Vivox (official UGS) or Dissonance	Photon Voice or Dissonance	Dissonance (official)	DIY
NAT traversal	Via relay transport	Unity Relay	Built in	Via relay transport	DIY — don't
Long-term suitability	✓ Very active, free forever	✓ Unity-backed	✓ But cloud-locked	✓ Mature, stable	✗ Risky

The disqualifier

Photon Fusion 2 cannot do offline LAN. Its connectivity model goes through Photon Cloud. Your #1 requirement — play on a hotspot with no internet — rules it out, despite it being the best netcode on the list. Same for PUN2. Don't get seduced by the tutorials.

Recommendation: **FishNet**

Use FishNet (free) + FishNet Pro (\$10 one-time) for lag compensation.

Why:

1. **Free client-side prediction.** The only free library with it built in. You will need it — a shooter without prediction feels like mud at 60 ms RTT, and hand-rolling reconciliation is a multi-week trap.
2. **Offline LAN is a first-class citizen.** Tugboat (LiteNetLib/UDP) connects to an IP. That's the whole story. No cloud, no account, no SDK init.
3. **One codebase, both modes.** Swap transports; **Multipass** even runs several at once.
4. **Cost.** Free core, ~\$10 one-time for Pro (optionally \$2/mo for ongoing updates). Compare to \$250/mo at 1,000 CCU on Photon.
5. **Actively developed** with a responsive Discord, which matters more than docs when you're stuck at 1am.

Trade-off, stated honestly: FishNet's documentation is thinner than Unity's and the prediction API has had breaking changes between major versions. Budget a week of confusion. If after two weeks of Phase 3 you're drowning, **Unity NGO 2.x is the acceptable fallback** — better docs, official Relay, official Vivox integration — at the cost of writing your own prediction or shipping with client-authoritative movement.

Do not write custom UDP networking. You'd spend 6+ months rebuilding what FishNet gives you free, and the result would be worse.

4. Local hotspot / LAN multiplayer (the important one)

4.1 The exact flow

Step 1 — Host enables the hotspot.

Be aware of the Android reality: **you cannot reliably create a normal tethering hotspot programmatically**. It needs system/carrier privileges. `WifiManager.startLocalOnlyHotspot()` exists (Android 8+) but auto-generates the SSID/password and requires `NEARBY_WIFI_DEVICES` on Android 13+.

So: **instruct the user**. Show a "Create Hotspot" screen with numbered steps and an `Open Wi-Fi Settings` button (an `Intent` deep link). This is exactly what Mini Militia did, and everyone already knows how.

Step 2 — Other players join the hotspot. Via OS Wi-Fi settings. *Nice touch:* the host screen shows a QR code encoding `WIFI:S:<ssid>;T:WPA;P:<password>;` — Android's camera app parses this natively and offers one-tap join. Cuts a frustrating 30-second step to 3 seconds.

Step 3 — Host creates a room. Taps "Create Room":

- FishNet `Tugboat` starts server on **UDP 7770** + local client
- `LanBeaconBroadcaster` starts broadcasting on **UDP 7771**, once per second

Beacon payload (~64 bytes, binary):

```
magic(4) 'MMB1' | protoVersion(2) | serverPort(2) | roomGuid(16)
| roomName(1+≤24 UTF8) | playerCount(1) | maxPlayers(1) | mapId(1) |
flags(1)
```

`flags = hasPassword | matchInProgress | allowsJoinInProgress.`

Broadcast to **both**:

- `255.255.255.255:7771` (limited broadcast)
- the subnet-directed broadcast derived from the local interface (e.g. `192.168.43.255`)

Some Android builds silently drop one or the other. Sending both costs 64 bytes/second.

Step 4 — Clients discover. "Join Game" screen:

- Acquire an Android **WifiManager.MulticastLock** (needs **CHANGE_WIFI_MULTICAST_STATE**). **Without this, many Android devices drop broadcast/multicast packets**, especially under power save. This is the single most common reason LAN discovery "mysteriously doesn't work." Release it when leaving the screen — it drains battery.
- Bind a **UdpClient** on 7771, **EnableBroadcast = true**
- Listen 3 s, dedupe by **roomGuid**, take the **sender IP from the datagram** (never try to guess or enumerate)
- Render a list: *"Rahul's Room · 3/6 · Foundry"*

Step 5 — Join. Tap → **Tugboat.SetClientAddress(ip, 7770) → StartConnection(Client)**. Handshake RPC: { **protoVersion, playerGuid, displayName, colorPref** }. Host validates and either assigns a slot or rejects with a typed reason (**VersionMismatch / RoomFull / Banned / MatchInProgress**) so the client can show a real message instead of "connection failed."

Step 6 — Start match. Host taps Start → **MatchState = Loading** → FishNet scene management loads the map on all → each client RPCs **SceneReady** → when all ready (or 10 s timeout), **Countdown** → **Playing**.

Step 7 — Sync. Host simulates authoritatively at 30 Hz. Clients predict their own player and interpolate remotes ~50 ms in the past. On LAN you'll see **2–8 ms RTT** — it will feel essentially local.

4.2 Fallbacks — build all three, you will need them

Broadcast discovery fails often enough in the wild (AP isolation on some routers, aggressive OEM power management, certain Android 14+ devices) that shipping without fallbacks guarantees 1-star reviews.

1. **Join by IP** — host displays its local IP in huge text; client types it. Ugly, always works.
2. **QR join** — host shows a QR encoding **ip:port:roomGuid**; client scans with an in-app scanner (ZXing.Net). **This is the best UX** — faster than picking from a list. Consider making it the primary path and the browse list secondary.
3. **Retry + manual rescan button** — never auto-fail silently.

4.3 What you need vs. don't

	Needed?	Notes
Dedicated server	✗ No	Zero servers on LAN. That's the whole point.
Host device	✓ Yes	One phone is server + client
UDP	✓ Yes	Both discovery and gameplay
TCP	✗ No	FishNet's reliable-UDP channel covers "must arrive" messages
Broadcast	✓ Yes	Limited + subnet-directed, both
Multicast	⚠ Optional	Broadcast is simpler and more reliable on Android hotspots. You still need the MulticastLock.
Local IP discovery	✓ Yes	Read the datagram's source address

	Needed?	Notes
NAT traversal	 No	Same subnet

4.4 What happens when the hotspot creator leaves

Three distinct cases — be precise about them:

Case A — hotspot owner *is* the game host (the normal case). The Wi-Fi network itself vanishes. Every client loses the interface. **Nothing can save this session.** Do this:

- Detect within 3 s (transport timeout + **NetworkReachability** check)
- Show "Connection lost — host left" with one big "Back to Rooms" button
- Cache names/colors/scores locally so re-hosting is instant
- No spinner, no 30-second timeout. Fail fast and loud.

Case B — hotspot owner $\neq$ game host, and the hotspot owner leaves. Identical outcome. The AP is gone. Same handling.

Case C — game host leaves but the hotspot stays up. This is the only migratable case. MVP answer: drop everyone to the room browser with a scan already running; the first person to tap "Create Room" becomes the new host. ~5 seconds, zero code beyond what you already have.

Prevention beats recovery here. In the lobby, show a small badge: "⚡ *Rahul is hosting — if Rahul leaves, the match ends.*" Players will self-organize around it.

5. Online multiplayer (Phase 7 — after LAN ships)

5.1 The flow

```

Android Client
  ↓ anonymous device auth (no login screen, no email)
Lobby Service – create/list/join rooms, join codes, room metadata
  ↓ host requests relay allocation, receives join code "X7K2QP"
Relay (cloud) – forwards UDP, no NAT traversal needed by you
  ↓
Host Client (authoritative sim) ↔ up to 5 joining clients
  ↓
Voice runs alongside on a separate cloud channel (EOS RTC) – NOT through
the host

```

Recommended provider for MVP-online: Epic Online Services — free at any scale, gives you anonymous Device ID auth + Lobbies + P2P with NAT punchthrough and relay fallback, works on Android via the PlayEveryWare Unity plugin. FishNet has an EOS transport. *Caveat found in the EOS issue tracker:* there are known mobile quirks around lobby joining and RTC on Android/iOS — budget testing time, and keep a second option ready. **Backup:** Unity Relay + Lobby (free tier $\approx$ 50 avg CCU, then metered) or Photon Fusion Cloud (free $\leq$ 100 CCU) used *only* for the online path while LAN keeps using Tugboat.

5.2 Practical netcode parameters

Parameter	Value	Why
Simulation tick	30 Hz	60 Hz doubles battery/thermal cost for imperceptible gain in a 2D shooter
Input send rate	30 Hz, 3 inputs/packet	Free packet-loss recovery
Snapshot rate	20 Hz online, 30 Hz LAN	2.6 KB/s down
Interpolation delay	100 ms online (2 snapshots), 50 ms LAN	Smooth remotes through jitter
Extrapolation cap	150 ms	Beyond this, players visibly teleport-correct
Max rewind (lag comp)	200 ms	Prevents high-ping players shooting around corners
Transport timeout	10 s	Mobile backgrounding needs headroom
Reconnect window	45 s	Long enough for a lift/tunnel, short enough not to hold slots

5.3 Server authority

Host owns: health, damage application, deaths, respawns, pickups, ammo, score, match timer, spawn point selection. **Client owns:** input intent only.

A client never says "I have 80 health" or "I killed Rahul." It says "I pressed fire at tick 4021 aiming 47°." Everything else is a host decision, broadcast back.

5.4 Client prediction, plainly

The local player must respond in **0 ms**, not in one round-trip. So:

1. Client applies input locally right now → stores **(tick, input, resultingState)** in a 64-entry ring buffer.
2. Host receives input, simulates, broadcasts authoritative state for tick T.
3. Client compares that to its stored state for tick T.
4. If the difference exceeds ~0.02 units → snap to the host state and **re-simulate ticks T+1 → current in a single frame** using the buffered inputs.
5. Smooth the visual correction over ~100 ms rather than snapping the sprite.

FishNet implements this via its replicate/reconcile methods. **Use the library's implementation.** Hand-rolling reconciliation is the single most common place solo multiplayer projects die.

5.5 Interpolation

Remote players are always rendered ~100 ms in the past, lerping between the last two received snapshots. You are trading 100 ms of staleness for perfectly smooth motion. This is the right trade, and it's what every shooter does.

5.6 Lag compensation

When a client fires at tick T , the host rewinds every other player's hitbox to where they were at tick T (from a ~500 ms history buffer), does the raycast, then restores. FishNet Pro's **ColliderRollback** does this.

For **projectile** weapons (rockets, grenades) you don't need rollback: spawn a local fake projectile instantly on the shooter's screen, spawn the real one host-side, and hide the fake when the real one replicates. FishNet documents this pattern and it works well on mobile.

MVP shortcut: ship with generous hitboxes and no rollback. For a chaotic 6-player jetpack brawl, nobody is frame-counting. Add rollback in Phase 7.

5.7 Packet loss

- State snapshots on the **unreliable** channel, each self-contained. A lost snapshot is simply skipped — the next one arrives 50 ms later and interpolation covers it.
- Inputs redundantly packed (3 per packet).
- **Reliable** channel *only* for: spawn/despawn, match state transitions, chat, roster changes, kill feed. Keep this list short — reliable ordered messages cause head-of-line blocking, which is what makes games hitch.
- Target: survive 5% loss with no visible artifacts. Test with a network conditioner (Clumsy on Windows, **dummysnet/pfctl** on macOS).

5.8 Cheating prevention — realistic expectations

With a player-host, **the host can cheat**. That's inherent to the model, and it's the price of \$0/month. Original Mini Militia was famously modded to death. Accept it for MVP, mitigate the cheap stuff:

Do now (a day's work, catches 90% of casual cheating):

- All damage/score/ammo server-side. Never accept **Health** from a client.
- Clamp input: $|\text{moveX}| \leq 1$, aim angle is inherently bounded, stick magnitude normalized server-side.
- Enforce fire-rate cooldowns host-side; reject shots inside the cooldown.
- Line-of-sight + max-range check on every claimed hit.
- Rate-limit RPCs per connection.

Do later: dedicated servers for ranked/public matches (removes host advantage entirely), server-side stat anomaly detection (KD > 20 flagged), IL2CPP + basic memory integrity. Don't buy anti-cheat — for a friends-on-a-hotspot game it's a non-problem, and for public online, dedicated servers are the real fix.

6. Voice chat — **online mode only**

Scope decision: voice ships with online multiplayer (Phase 8), not with the LAN/hotspot MVP. Rationale and consequences below.

6.1 Why LAN mode has no voice

Players on the same hotspot are, by the physics of Wi-Fi range, within about 30 metres of each other — a classroom, a dorm room, a bus, an office. **They can already hear each other.** Adding a voice channel there buys almost nothing and costs a lot:

- Six phones on speaker in one room is an echo and feedback nightmare — the worst possible acoustic environment for a voice system.
- Every local mic re-broadcasts the same room audio back into the mix.
- It forces **RECORD_AUDIO** into your v1 permission set and your Play Store data-safety declaration.
- It puts a 75 KB/s relay burden on the host phone, the most performance-constrained device in the session.

Cutting it removes ~2 weeks from the pre-launch roadmap and makes your first release meaningfully simpler to ship and to review.

6.2 What this unlocks

The original constraint — *voice must work with no internet* — is what forced voice onto your own game transport, because every cloud voice service routes audio through its own servers. **Removing that constraint removes the constraint on the solution.** Once voice only exists in online mode, an internet connection is guaranteed, and a cloud voice service becomes the better engineering choice:

	Voice on your transport (Dissonance)	Voice on a cloud service (EOS RTC / Vivox)
Works offline on LAN	✅	❌ (irrelevant now)
Load on the host phone	+75 KB/s up, host relays every stream	Zero — host never sees voice
Echo cancellation / noise suppression	Good	Better (purpose-built, tuned for mobile)
Upfront cost	\$120	\$0
Cost at 10k CCU	~\$0	Metered (EOS RTC free; Vivox metered)
Integration effort	Low	Low

The second row is the important one. §12's risk #5 is that the host phone becomes the bottleneck, and voice relay was the single largest contributor to that. Moving voice to a cloud channel **deletes that risk** rather than mitigating it.

6.3 How real-time voice works

```

Microphone (16 kHz mono)
  → 20 ms frames (320 samples)
  → gate: push-to-talk OR voice activity detection
  → Opus encode (~16–24 kbps)
  → unreliable packets → voice service (NOT your game host)
  → fan-out to the other players in the voice room

```

```

→ adaptive jitter buffer (60–200 ms)
→ Opus decode
→ AudioSource playback
+ acoustic echo cancellation & noise suppression

```

6.4 Should voice share the gameplay networking infrastructure?

No — and this is now a deliberate separation rather than a compromise.

Gameplay and voice have opposite requirements. Gameplay is tiny, extremely latency-sensitive, and must be authoritative. Voice is bandwidth-heavy, tolerant of 150–250 ms, and needs no authority at all.

Running them on separate channels means:

- A voice bandwidth spike can never degrade gameplay tick delivery.
- The host phone isn't taxed by traffic it has no reason to touch.
- Voice rooms can outlive a match (lobby banter, rematch voting) independent of the game session lifecycle.

The two are coordinated only by shared identity: the same **MatchId** names the voice room, and the same **playerGuid** identifies a speaker.

6.5 Options

	Cost	Effort	Notes
EOS RTC ★	Free	Low	Vivox-powered, bundled with the EOS stack \$5 already recommends for auth + lobby + relay. One SDK, one account system, no extra vendor. Known Android/iOS quirks around lobby-joined RTC — budget testing time.
Vivox (UGS)	Free tier, then metered	Low	The mature choice if you go Unity Gaming Services for online instead of EOS. Excellent mobile AEC. Official Unity package.
Dissonance	\$120 one-time	Low	Still the right answer <i>if you ever decide you want LAN voice after all</i> — it's the only option that works offline. Also a solid fallback if EOS RTC's Android issues block you.
Photon Voice 2	Photon CCU pricing	Low	Only makes sense if you're already paying Photon for gameplay. You're not.
Agora / 100ms	Metered, scales expensively	Low	Overkill. Built for large-scale broadcast, priced accordingly.
Raw WebRTC	Free	Very high	Signaling server, mesh of 30 peer connections at 6 players, painful Unity integration. No.

6.6 Recommendation: **EOS RTC**, with Dissonance as the fallback

Use EOS RTC. If you follow §5 and adopt Epic Online Services for anonymous auth, lobbies and relay, voice is already in the SDK you've integrated — same initialization, same identity, same lobby object. It is free at any scale, it keeps voice traffic entirely off the host, and it gives you mobile-tuned echo cancellation you would otherwise have to tune yourself.

Keep Dissonance in your back pocket for two scenarios: EOS RTC's known Android RTC quirks turn out to be blocking, or you later decide LAN voice is worth having after all. It's a \$120 purchase you can make at that point, not now.

If you choose Unity Gaming Services over EOS for the online backend, use **Vivox** instead — same reasoning, and it's the first-party option there.

6.7 Push-to-talk vs always-on

Default: latching push-to-talk (tap to open the mic, tap to close).

Not *hold*-to-talk — both thumbs are already occupied by the movement and aim sticks, so there is no free thumb to hold a button with. A latching toggle in the top-center of the screen with a clear "MIC LIVE" indicator is the right mobile ergonomic. Offer open-mic with VAD as a settings toggle for headset users.

Why PTT by default:

- Bounded bandwidth and predictable cost
- No background noise from five idle mics in five different environments
- Fewer accidental hot-mic moments
- Clear, explicit user control over a sensitive permission

Note that the worst echo scenario — everyone in one room on speaker — is largely designed out already, because voice only exists in online mode where players are geographically apart.

6.8 Handling 2–6 players

Six participants is small enough that fan-out topology barely matters, and with a cloud service it isn't your problem at all — you join a room and the service handles distribution.

- Cap **concurrent active speakers at 3** (loudest wins). Beyond three simultaneous talkers nobody can follow the conversation anyway, and it bounds the worst case cleanly.
- Render a small speaking indicator next to each player's name in the HUD and scoreboard so people can tell who is talking.

6.9 Latency and bandwidth

Latency: expect **100–250 ms** end-to-end through a cloud service. That is entirely fine for conversation — it is not a competitive callout system, it's people laughing at each other. Do not spend Phase 8 chasing it lower.

Bandwidth per player: Opus at 16 kHz mono $\approx$ **3 KB/s (24 kbps)** up while speaking, plus up to 3×3 KB/s $\approx$ **9 KB/s** down when others talk. Call it **$\sim$ 12 KB/s peak per player**.

Compare to gameplay at $\sim$ 3.6 KB/s per player: **voice is roughly 3–4× your gameplay traffic even with a 3-speaker cap**. The difference from the earlier design is *where* that traffic goes — to the voice provider,

not through a player's phone.

6.10 Rooms, muting, joining and leaving

- **One voice room per match**, named by `MatchId`. Join it on entering the online lobby, leave it on returning to the main menu. Keep it alive through `MatchEnd` → `RematchVote` → `next match` — the between-round banter is a large part of why people keep playing.
- **Mute is client-side**: stop rendering that participant's audio. Instant, no round-trip, and persist it per `playerGuid` in `PlayerPrefs` so a consistently annoying friend stays muted across sessions.
- Put mute controls on the **scoreboard row** (tap a player → mute) and in the in-match player list. Don't bury them in a settings menu.
- **Leave/rejoin**: on disconnect, leave the voice room; on successful reconnect within the 45 s window, rejoin the same room by `MatchId`. Tie it to the same `ReconnectController` that restores the gameplay slot.
- Later, if you add teams: team rooms as `match_<id>_team_<n>`, with a toggle for all-chat.

6.11 Can voice work over local hotspot without internet?

Not with the recommended stack, and that is the intended design. EOS RTC and Vivox both route audio through their servers, so they require connectivity. LAN mode has no voice.

If you change your mind later, the path is well-defined and doesn't invalidate anything else in this plan: buy Dissonance (~\$120), which rides your existing FishNet transport and therefore works with no internet at all. It would slot into the same `Voice/` assembly behind the same interface. Budget 1–2 weeks. Design the `IVoiceSession` interface in Phase 8 with this in mind — `Join(roomId)`, `Leave()`, `SetMuted(playerGuid, bool)`, `SetSelfTransmitting(bool)` — and swapping providers stays a contained change.

6.12 Android voice specifics (Phase 8)

- **RECORD_AUDIO** runtime permission — show a rationale screen first ("so you can talk to your teammates"), and let people decline and keep playing. **Never block match entry on it.**
- This permission does **not** exist in your v1 LAN build, which keeps the initial Play Store data-safety declaration simple. It arrives with the online update.
- Handle audio focus: an incoming call must suspend capture and playback, then resume cleanly.
- Persistent on-screen mic indicator whenever the mic is live.
- Test on both speaker and headphones, and specifically test a player who denies the permission mid-session.

7. MVP scope (Version 1)

Build this

Core loop

- 1 game mode: free-for-all deathmatch
- 1 map: small, symmetric, ~2 screens wide, 3 platform tiers
- 2–6 players

- Match end: 5-minute timer **or** first to 15 kills

Movement & combat

- Run left/right, jump, **jetpack with fuel gauge** (the signature mechanic — get this feeling right above all else)
- Right stick aims + auto-fires (Mini Militia style — no separate fire button)
- Default pistol (infinite ammo) + 2 pickup weapons (rifle, shotgun)
- Grenades: 1 button, 3 per life
- 100 HP, damage, death, 3 s respawn, 1.5 s spawn invulnerability

Session

- Create Room / Browse LAN / Join by IP / Join by QR
- Lobby with ready-up, name entry, color pick
- Scoreboard on match end
- **Rematch** — "Play Again," 10 s vote, session never torn down
- Reconnect within 45 s with score restored
- Kill feed, match timer, health bar, fuel gauge

Settings

- Stick sensitivity, SFX/music volume

No voice chat in v1. It ships with online mode in Phase 8. See §6.1 — hotspot players are in the same room and can already talk, and leaving **RECORD_AUDIO** out keeps your first release and its data-safety declaration simple.

Do NOT build this in v1

Category	Skip
Networking	Online multiplayer, matchmaking, host migration, dedicated servers, anti-cheat
Voice	Voice chat of any kind — no mic capture, no RECORD_AUDIO permission (arrives in Phase 8 with online)
Accounts	Login, profiles, cloud saves, friends lists, clans
Progression	XP, levels, unlocks, battle pass, ranked/MMR, leaderboards
Monetization	IAP, ads, cosmetics, currency
Content	More maps, more modes (CTF/team/survival), vehicles, melee, crouch/prone, destructible terrain, bots/AI
Social	Text chat, emotes, spectator mode, replays
Platform	iOS, tablets-specific UI, controller support, localization
Polish	Map editor, tutorials beyond a hint overlay, push notifications

The temptation you must resist: building online multiplayer before shipping LAN. LAN is your differentiator, costs \$0/month to operate, requires no backend, and is testable with two phones on your

desk. Ship it. Get real players. *Then* decide if online is worth the money and complexity.

8. Development roadmap

Estimates assume a **solo dev**, **~12–15 hrs/week**, comfortable with backend but new to Unity. Halve them if full-time.

Phase 0 — Unity fundamentals · 2–3 weeks · Low complexity

Build: Throwaway single-player scene — a sprite that runs, jumps, and shoots at static targets. On-screen virtual joysticks. Deploy to a physical Android device on day 3, not week 3.

Learn: Unity 6 Editor, GameObject/Component model, prefabs, **Update** vs **FixedUpdate**, 2D URP, Tilemap, Sprite Renderer, **Physics2D.BoxCast**, new Input System + On-Screen Controls, **ScriptableObject**, Android build pipeline (SDK/NDK, IL2CPP, ARM64, keystore).

Depends on: nothing.

Done when: an APK on your phone lets you move, jump and shoot with touch controls at a stable 60 fps.

Phase 1 — Offline prototype (game feel) · 3–4 weeks · Medium

Build: The complete single-player game against dumb targets. Kinematic **PlayerMotor** with gravity + jetpack + fuel. Aim/auto-fire. Hitscan + projectile weapons. Health, death, respawn. The real map. Camera. HUD. Placeholder art — literally colored rectangles is fine.

Learn: Writing a deterministic kinematic controller, object pooling, **ScriptableObject**-driven weapon definitions, animation via sprite sheets, game feel (screenshake, hit-stop, recoil, muzzle flash, tracers).

Depends on: Phase 0.

Done when: you play it for 10 minutes alone and it's *already fun*. **This is a hard gate.** If the jetpack doesn't feel good solo, no amount of netcode will save it. Get a friend to play; watch their face, not the framerate.

Critical architectural discipline in this phase: **PlayerMotor** must be pure C#, take an input struct, and have zero Unity-networking references. Everything in Phase 2 depends on this.

Phase 2 — Multiplayer networking core · 3–4 weeks · High — the hardest phase

Build: Install FishNet. Convert the player to a **NetworkObject**. Host + client in the same room over localhost/desktop Wi-Fi (use **ParrelSync** to run two Editor instances). Input replication, host-authoritative movement, prediction/reconciliation, remote interpolation. Host-authoritative damage, death, respawn, score.

Learn: Client-server topology, ticks, **SyncVars**, RPCs (Server/Observers/Target), FishNet's prediction API, reliable vs unreliable channels, why you must never trust the client.

Depends on: Phase 1's clean simulation layer.

Done when: two Editor instances + one phone play a 3-player deathmatch on your home Wi-Fi with smooth movement, and with 150 ms artificial latency injected the local player still feels responsive.

Budget for frustration here. Reconciliation bugs (rubber-banding, jitter, desync) are the classic multiplayer time sink. Build a debug overlay early: RTT, tick offset, reconcile count/sec, packets in/out. You cannot fix what you can't see.

Phase 3 — LAN / hotspot multiplayer · 2–3 weeks · Medium

Build: `LanBeaconBroadcaster` + `LanBeaconListener` (UDP 7771). Android `MulticastLock` plugin wrapper. Room browser UI. Join-by-IP. QR host/scan. Hotspot instruction screen with settings deep link. Version handshake with typed rejection reasons.

Learn: UDP broadcast/directed broadcast, `System.Net.Sockets` in Unity, Android manifest permissions (`INTERNET`, `ACCESS_NETWORK_STATE`, `CHANGE_WIFI_MULTICAST_STATE`, `NEARBY_WIFI_DEVICES` if you touch Wi-Fi APIs), `AndroidJavaObject` interop.

Depends on: Phase 2.

Done when: Phone A creates a hotspot from OS settings and hosts; Phones B and C join the hotspot, see the room appear within 3 s, join, and play a full match — **with the router unplugged and mobile data off**. Test on at least 3 different OEMs (Samsung, Xiaomi, and one other — their power management differs wildly).

Phase 4 — Full match loop · 3–4 weeks · Medium

Build: Lobby with ready-up, names, colors. `MatchRoster` + `PlayerSlot` keyed by `playerGuid`. Countdown → Playing → MatchEnd → RematchVote. Scoreboard. Kill feed. Disconnect handling + 45 s reconnect. Host-left → back-to-browser flow. `OnApplicationPause` handling.

Learn: State machine design over a network, scene management with FishNet, late-join, identity vs connection separation.

Depends on: Phase 3.

Done when: you can play 5 consecutive matches without leaving the session; killing one client's Wi-Fi for 20 s and restoring it rejoins them with score intact; and the host leaving produces a clean 5-second path back into a new match.

Phase 5 — Polish & optimization · 3–4 weeks · Medium

Build: Real art and animation. Sound design. Screenshake, hit-stop, particles, kill effects. UI pass. Settings. Performance: target 60 fps on a 4-year-old mid-range phone; profile with the Unity Profiler *on device*; sprite atlasing; draw-call reduction; GC allocation elimination in the game loop (this is the #1 mobile stutter cause). Battery/thermal check: 20-minute session without the phone becoming a hand-warmer.

Learn: Unity Profiler and Memory Profiler on Android, sprite atlases, addressables (optional), URP mobile settings, GC pressure.

Depends on: Phase 4.

Done when: stable 60 fps on your oldest test device through a full 5-minute match with 6 players; APK under ~80 MB; no frame spikes above 33 ms.

Phase 6 — Android release (LAN-only v1.0) · 2 weeks · Low

Build: App icon, store listing, screenshots, privacy policy, data safety form, keystore management, AAB build, internal test track → closed beta (20–50 people) → production.

Learn: Play Console, target API level requirements, AAB vs APK, staged rollout, Crashlytics/Cloud Diagnostics.

Depends on: Phase 5.

Simplified by the no-voice decision: v1 requests no `RECORD_AUDIO` permission, so your data-safety declaration has no audio collection to disclose and your privacy policy is correspondingly shorter. One less review surface on your first submission.

Done when: live on Google Play, \$0/month operating cost, and you have real crash-free-rate data.

Ship here. Stop. Gather feedback for 4+ weeks before Phase 7.

Phase 7 — Online multiplayer · 4–6 weeks · High

Build: `ITransportProvider` implementation for relay. Anonymous device auth (EOS Device ID). Lobby create/list/join + join codes. Quick Match (join any room with space, else create). Region selection. Online-tuned interpolation. Lag compensation (FishNet Pro `ColliderRollback`). Server-side sanity clamps. Reconnect via join code.

Learn: Relay allocations, lobby APIs, NAT realities, region latency, the CCU/bandwidth billing model.

Depends on: Phase 6 and real evidence that people want it.

Done when: two players on different mobile networks in different cities play a full match at <120 ms RTT and reconnect successfully after a 20 s network drop.

Phase 8 — Voice chat (online only) · 1–2 weeks · Low–Medium

Build: `IVoiceSession` interface (`Join` / `Leave` / `SetMuted` / `SetSelfTransmitting`) with an EOS RTC implementation behind it. Voice room keyed by `MatchId`, joined on entering the online lobby and held through rematch. Latching PTT button + "MIC LIVE" indicator. Speaking indicators in HUD and scoreboard. Per-player mute persisted by `playerGuid`. 3-concurrent-speaker cap. `RECORD_AUDIO` rationale flow with graceful decline. Audio focus handling for incoming calls. Voice room leave/rejoin wired into `ReconnectController`.

Learn: EOS RTC lifecycle, PTT vs VAD, jitter buffers, Android audio focus and routing, mic permission UX.

Depends on: Phase 7 — voice is online-only, so it cannot be built before online exists.

Done when: two players on different mobile networks hold a conversation through a full match and a rematch without dropping the room; muting is instant and survives an app restart; a player who denies the mic permission can still play normally; and an incoming call suspends and resumes voice cleanly.

Keep this phase shippable on its own. Online without voice is a perfectly good release; voice is an additive update on top of it.

Total to LAN-only Play Store launch: ~4.5–5.5 months part-time — about two weeks shorter than it would be with voice in the MVP. Online adds ~1.5 months; voice adds ~2 weeks on top of that.

9. Project structure

```
Assets/
├── _Project/                                # everything you author lives under
one root
├── Art/                                     Sprites/ Animations/ VFX/ Fonts/ Atlases/
├── Audio/                                  SFX/ Music/ Mixers/
├── Maps/                                   Tilemaps/ Palettes/ MapDefinitions/
(ScriptableObjects)
├── Prefabs/                               Player/ Weapons/ Projectiles/ Pickups/ UI/ Network/
├── Scenes/                                00_Boot 10_MainMenu 20_Lobby 30_Map_Foundry
├── Settings/                              URP assets, InputActions, Quality, BuildProfiles
├── Data/                                   WeaponDefinition, GameModeDefinition, BalanceConfig
(SOs)
├── Scripts/
│   ├── Core/                             # Core.asmdef – ZERO dependencies
│   │   ServiceLocator, GameEventBus, TickTimer, ObjectPool<T>,
│   │   Log, MathUtils, Quantize
│   ├── Config/                           # Config.asmdef – deps: Core
│   │   NetworkConstants (ports, magic, protocol version),
GameplayConstants
│   ├── Gameplay/                         # Gameplay.asmdef – deps: Core, Config
│   │   └── Player/                       # ⚠ MUST NOT reference FishNet
│   │       PlayerMotor, JetpackSim, PlayerState,
PlayerInput
│   │       Weapons/                     WeaponSim, HitscanSim, ProjectileSim,
Inventory
│   │       Combat/                      DamageResolver, Hitbox, HitResult
│   │       Pickups/                     PickupSim, PickupSpawner
│   │       Rules/                       MatchRules, ScoreRules, RespawnRules
│   │       Map/                         SpawnPointRegistry, MapBounds
└── Networking/                           # Networking.asmdef – deps: Core, Config,
```

```

Gameplay, FishNet
|
|   Session/
|   NetworkBootstrap, SessionManager,
|   ITransportProvider, LanTransport,
RelayTransport
|
|   Discovery/
|   ISessionDiscovery, LanBeaconBroadcaster,
|   LanBeaconListener, BeaconPacket, RoomInfo
|   Replication/
|   NetworkPlayer, NetworkPlayerMotor
(rePLICATE/reconcile),
|
|   NetworkWeapon, NetworkProjectile,
SnapshotBuffer
|
|   Match/
|   MatchStateMachine, MatchRoster, PlayerSlot,
|   ScoreTracker, RespawnService, KillFeedRelay
|   Identity/
|   PlayerIdentity (guid), ReconnectController
|   Validation/
|   InputSanitizer, RateLimiter, HitValidator
|   Online/
|   (Phase 7) LobbyProvider, RelayProvider,
Matchmaker, AuthService
|
|   Voice/
|   # Voice.asmdef – Phase 8, ONLINE ONLY. deps:
Core, Networking, EOS
|   IVoiceSession, EosRtcVoiceSession, PttController,
|   MuteRegistry, MicPermissionFlow, SpeakingIndicators
|
|   UI/
|   # UI.asmdef – deps: Core, Config; reads
Networking via events only
|   Screens/
|   MainMenu, HostSetup, RoomBrowser, JoinByIp,
QrJoin,
|   Lobby, Hud, Scoreboard, RematchVote,
Settings
|   Widgets/
|   VirtualJoystick, HealthBar, FuelGauge,
KillFeed,
|   MicIndicator, PlayerRow
|
|   Platform/
|   # Platform.asmdef – deps: Core
|   AndroidMulticastLock, WifiSettingsIntent, QrScanner,
LocalIpResolver
|
|   Editor/
|   # Editor.asmdef
|   BuildScripts, ParrelSyncHelpers, BalanceInspector
|
|   Plugins/Android/
|   AndroidManifest.xml, .aar files
|   ThirdParty/
|   FishNet/ ZXing/ EOS-SDK/ (Phase 7+)

```

Assembly definitions matter here. They cut compile times from ~40 s to ~5 s (enormous when you're iterating on netcode), and more importantly they *enforce* the dependency direction. If `Gameplay.asmdef` doesn't reference `FishNet`, you physically cannot accidentally couple your simulation to your networking library. That constraint is worth more than any amount of discipline.

Where each thing lives, per your list:

Concern	Location
Player	<code>Gameplay/Player</code> (sim) + <code>Networking/Replication/NetworkPlayer</code> (wrapper)

Concern	Location
Weapons	Gameplay/Weapons (sim) + Data/WeaponDefinition (balance SOs) + Prefabs/Weapons
Network	Networking/ — session, discovery, replication, match, validation
Lobby	Networking/Match (state) + UI/Screens/Lobby (view)
Voice	Voice/ — empty until Phase 8; online builds only
UI	UI/ — never references FishNet directly; subscribes to GameEventBus
Game state	Networking/Match/MatchStateMachine — single source of truth, host-owned
Maps	Maps/ + Gameplay/Map
Audio	Audio/ assets + a Core -level AudioService
Matchmaking	Networking/Online — doesn't exist until Phase 7

10. Backend requirements

Required for MVP: **none**

This is the best strategic property of a LAN-first game. **Zero servers, zero monthly cost, zero ops, zero GDPR surface.** Your v1.0 ships with no backend at all.

Add these two anyway, both free:

- **Crash reporting** — Firebase Crashlytics or Unity Cloud Diagnostics. Non-negotiable; you cannot debug OEM-specific crashes without it.
- **Basic analytics** — Unity Analytics or Firebase. Track only: match started, match completed, player count, match duration, crash-free rate, discovery-method-used (broadcast/IP/QR — this tells you whether your LAN discovery actually works in the wild).

Phase 7 (online) — required then

Service	Choice
Anonymous auth	EOS Device ID (free) or Unity Auth — no login screen
Lobby / room list	EOS Lobbies or Unity Lobby
Relay	EOS P2P or Unity Relay
Voice	EOS RTC — free, bundled in the same SDK, separate channel from gameplay

Can be added later (and mostly shouldn't be)

Player profiles, leaderboards, friends, progression/XP, cosmetics & economy, remote config, push notifications, dedicated servers, anti-cheat, replay storage, clan systems.

Each of these is a permanent operational commitment. A leaderboard is a database, a migration path, a moderation problem, and an abuse vector. Add them only when a specific, measured player demand justifies the cost.

11. Cost analysis

Figures are approximate as of Sept 2026 and vendor pricing changes — verify against current pricing pages before committing. They're accurate to order of magnitude.

Development & testing (one-time)

Item	Cost
Unity Personal (free under \$200k revenue, no Runtime Fee)	\$0
FishNet (free core)	\$0
FishNet Pro (lag compensation)	~ \$10 one-time (+\$2/mo optional for updates)
Voice chat (EOS RTC, Phase 8)	\$0
Google Play developer account	\$25 one-time
Test devices (2–3 cheap/used Androids; borrow friends' phones too)	\$0–400
Art & audio (Asset Store packs or a freelancer)	\$0–400
(<i>optional</i>) Dissonance, only if you later want LAN voice	~\$120
Total	≈ \$35–835 one-time

Plus ~4.5–5.5 months of your evenings. That's the real cost. Dropping voice from the MVP removed the single largest one-time software purchase from your pre-launch budget.

LAN-only version (v1.0)

Servers	\$0
Bandwidth	\$0
Voice	N/A — no voice in LAN mode
Monthly total	\$0

Unlimited players, forever, at zero marginal cost. This is a genuinely strong position — most multiplayer indies can't ship without a credit card attached.

Small online beta (~100 peak CCU)

Option	Monthly
EOS (auth + lobbies + P2P relay)	\$0
Photon Fusion free tier (≤ 100 CCU)	\$0 — or the 200 CCU Plus bundle at $\sim \$95/\text{yr} \approx \$8/\text{mo}$
Unity Relay + Lobby free tier (~ 50 avg CCU)	\$0 up to the cap
Voice (EOS RTC)	\$0
Realistic	\$0–15/month

1,000 peak CCU (≈ 170 concurrent 6-player matches)

Understand the billing model first: **CCU plans are priced on your peak; bandwidth allowances are consumed at your average.** With a typical 3–4 $\times$ peak-to-average ratio, 1,000 peak CCU is ~ 250 –350 average CCU.

Approach	Monthly
Photon Fusion Cloud (1,000 CCU, includes 3 GB/CCU/mo)	$\sim \\$250$
Unity Relay (metered bandwidth)	$\sim \\$150$–400 depending on actual traffic
EOS P2P relay	$\sim \\$0$ (but validate their fair-use terms at this scale)
Voice via EOS RTC (separate channel, off your relay)	\$0
Voice via Vivox/Agora instead	+\$300–800
Dedicated servers (Edgegap, pay-per-minute) instead of host mode	$\sim \\$800$–2,000
Recommended path (relay host + EOS RTC voice)	$\approx \\$150$–400/month

Note that voice no longer appears in your relay bandwidth at all — it travels on the provider's own network. That is worth roughly **$\$100$ –200/month at this scale** versus the earlier transport-riding design, and more importantly it takes the load off player phones.

10,000 peak CCU ($\approx 1,700$ concurrent matches)

Approach	Monthly
Photon Premium (linear $\sim \$0.50/\text{CCU}$ above 2,000)	$\sim \\$5,000$
Managed relay (gameplay only — voice is separate)	$\sim \\$1,500$–3,500
Dedicated servers (managed orchestration)	$\sim \\$6,000$–15,000
Voice: EOS RTC	\$0 — the main reason to prefer it over Vivox/Agora at scale
Voice: Vivox/Agora instead	+\$2,000–8,000

Approach	Monthly
Self-hosted relay on bare metal (e.g. 2–3 unmetered 1 Gbps boxes, ~\$50–80 each)	~\$150–400 + ops time

Where the money actually goes

1. **Voice is the #1 variable you control** — it is 3–4× your gameplay traffic. Because voice is online-only here, you get to pick a provider, and the choice is worth thousands per month at scale: **EOS RTC is free, Vivox and Agora are metered**. Pick the free one unless it fails you technically. Either way, voice never touches your relay bill or a player's phone.
2. **Dedicated servers are the #1 fixed cost** — 3–10× a relay model. Only adopt them when competitive integrity demands it (ranked mode), not by default.
3. **Managed convenience carries a 10–30× premium over bare metal**. At 10k CCU, self-hosting relay on unmetered dedicated servers is dramatically cheaper — but it's real ops work. That's your escape hatch if you ever get big, not a starting point.
4. **Gameplay bandwidth is free**. At 3.6 KB/s per player you will never be billed meaningfully for game state. Don't optimize it past the quantization you're already doing.

12. The 10 biggest technical risks

1. Android LAN discovery failing on real devices — *highest risk*

Why it's hard: Broadcast packets are silently dropped by OEM power management, AP client isolation, and Android's tightening Wi-Fi permissions. Behavior differs across Samsung / Xiaomi / OnePlus / Pixel. It'll work perfectly on your two phones and fail for 20% of users.

Reduce it: Acquire a **MulticastLock** (this alone fixes most cases). Broadcast to both **255.255.255.255** and the subnet-directed address. **Ship join-by-IP and QR-join from day one** — treat QR as a first-class path, not a fallback. Test on ≥4 OEMs. Add an analytics event recording which discovery method succeeded so you learn the real-world failure rate.

When: **MVP**. This is your core feature. It cannot be deferred.

2. Client prediction / reconciliation bugs (rubber-banding)

Why it's hard: Prediction is the hardest concept in game networking. Symptoms are non-local and intermittent — a float divergence in your motor manifests as a random teleport 30 seconds later. Hardest thing to debug in the whole project.

Reduce it: Use FishNet's built-in prediction rather than hand-rolling. Keep **PlayerMotor** pure and deterministic (no **Random**, no **Time.deltaTime** inside, no RigidBody2D). Build a debug overlay in week one showing RTT, tick offset, reconciles/sec, and predicted-vs-authoritative divergence. Test with injected latency and packet loss from the very first day of Phase 2 — never test only on LAN.

When: **MVP**. Movement feel is the game.

3. EOS RTC voice on Android — severity dropped sharply now that voice is online-only

Why it's hard: Epic's own release notes flag known issues with lobby-joined RTC on Android and iOS. Mobile audio is also fragmented territory — routing, audio focus, and permission behaviour differ across OEM skins. You are depending on a third party's mobile implementation.

Note on what changed: scoping voice to online mode removed the two worst parts of this risk. The old design put voice on the host phone's uplink (a bandwidth and CPU tax on the weakest device in the session) and put six open mics in one physical room (a guaranteed feedback loop). Both are now designed out rather than mitigated.

Reduce it: Build behind an `IVoiceSession` interface from day one so the provider is swappable. Prototype EOS RTC on two real Android devices in the *first week* of Phase 8, before wiring it into the UI. If it fails, switch to Vivox (metered but mature) or Dissonance (\$120, rides your transport, also gives you LAN voice). Default to latching PTT. Accept 100–250 ms — don't chase lower.

When: **Phase 8, well after launch.** This risk cannot block your v1 release, which is precisely why the scope change is valuable.

4. Mobile app lifecycle killing connections

Why it's hard: A phone call, a notification tap, or the screen locking pauses Unity. Your connection times out and the player is kicked mid-match. Happens constantly in real-world play and looks like "the game is broken."

Reduce it: Hook `OnApplicationPause/OnApplicationFocus`; send a `Suspended` hint before pausing. Set transport timeouts to ~10 s (not desktop defaults). Reserve the slot for 45 s. Auto-reconnect with backoff on resume. Keep a `MulticastLock/WifiLock` where appropriate. Test by literally calling your own test phone mid-match.

When: **MVP** (Phase 4).

5. Host device performance becoming the bottleneck

Why it's hard: The host runs the authoritative simulation *and* renders its own game. On a budget phone, the host's frame drops become everyone's lag.

Substantially reduced by the voice decision: the host previously also had to relay up to 75 KB/s of voice to five peers. With voice on a separate cloud channel, **the host never touches voice traffic at all** — the largest single contributor to this risk is gone, not merely mitigated.

Reduce it further: Keep the sim at 30 Hz. Zero GC allocations in the network loop (pool everything). Profile a host build on your *slowest* device, not your fastest. Show a "recommended host" hint. Consider a soft device check that warns very low-end phones before they host.

When: **MVP** — profile in Phase 5 at the latest.

6. Thermal throttling and battery drain

Why it's hard: 60 fps sustained over Wi-Fi means a hot device and a visible FPS cliff at ~10 minutes. Users blame the game. (v1 at least has no mic capture adding to the thermal load — that arrives with online in Phase 8.)

Reduce it: 30 Hz sim (not 60). Cap render to 60 fps and consider 30 as a battery option. Sprite atlasing to cut draw calls. Eliminate per-frame allocations. Short 5-minute matches naturally help. **Test a 20-minute continuous session** — not a 2-minute one — and watch the frame graph.

When: Phase 5, but design for it from Phase 1 (this is why 30 Hz is chosen up front).

7. Android device fragmentation

Why it's hard: OEM skins alter Wi-Fi, audio routing, background policy and permission behavior in undocumented ways. You cannot test them all solo.

Reduce it: Buy or borrow 3–4 devices spanning OEMs and Android versions (one should be ~4 years old and cheap). Ship Crashlytics before your first beta tester touches it. Run a closed beta of 20–50 people with varied phones for at least two weeks. Consider a cloud device farm for one broad sweep before launch.

When: Phase 6 for breadth, but get a second non-Pixel/non-Samsung device by Phase 3.

8. Host-side cheating once online launches

Why it's hard: In a player-host model, the host controls the authoritative simulation. Mini Militia was comprehensively modded. APKs get decompiled and reposted.

Reduce it: MVP is LAN-only among friends — cheating is socially policed and basically a non-issue. For online: server-side clamps on movement/fire-rate/ammo, interest management (don't send state a client shouldn't see — kills most wallhacks), rate limiting, IL2CPP. Move ranked play to dedicated servers when it matters. **Do not buy anti-cheat.**

When: Basic validation in MVP (cheap). Everything else at Phase 7+.

9. Reconnection and identity correctness

Why it's hard: Every naive implementation keys off the connection ID, which changes on reconnect. Result: duplicate players, lost scores, ghost avatars, slots leaking until the room is "full" with nobody in it.

Reduce it: **playerGuid** from day one; connection IDs are transient plumbing, never identity. Slot lifecycle (**Empty/Connected/Disconnected/Reserved**) as an explicit state machine, not booleans. Write integration tests that disconnect and rejoin 20 times in a loop and assert the roster is still correct.

When: **MVP** (Phase 4). Retrofitting identity later is a painful refactor.

10. Scope creep and solo-dev attrition — the risk most likely to actually kill this project

Why it's hard: Multiplayer games have an enormous "just one more feature" gravity. The failure mode isn't a technical wall — it's month 9 with an unshipped, 80%-done game and no motivation left. This kills far

more indie multiplayer projects than netcode ever does.

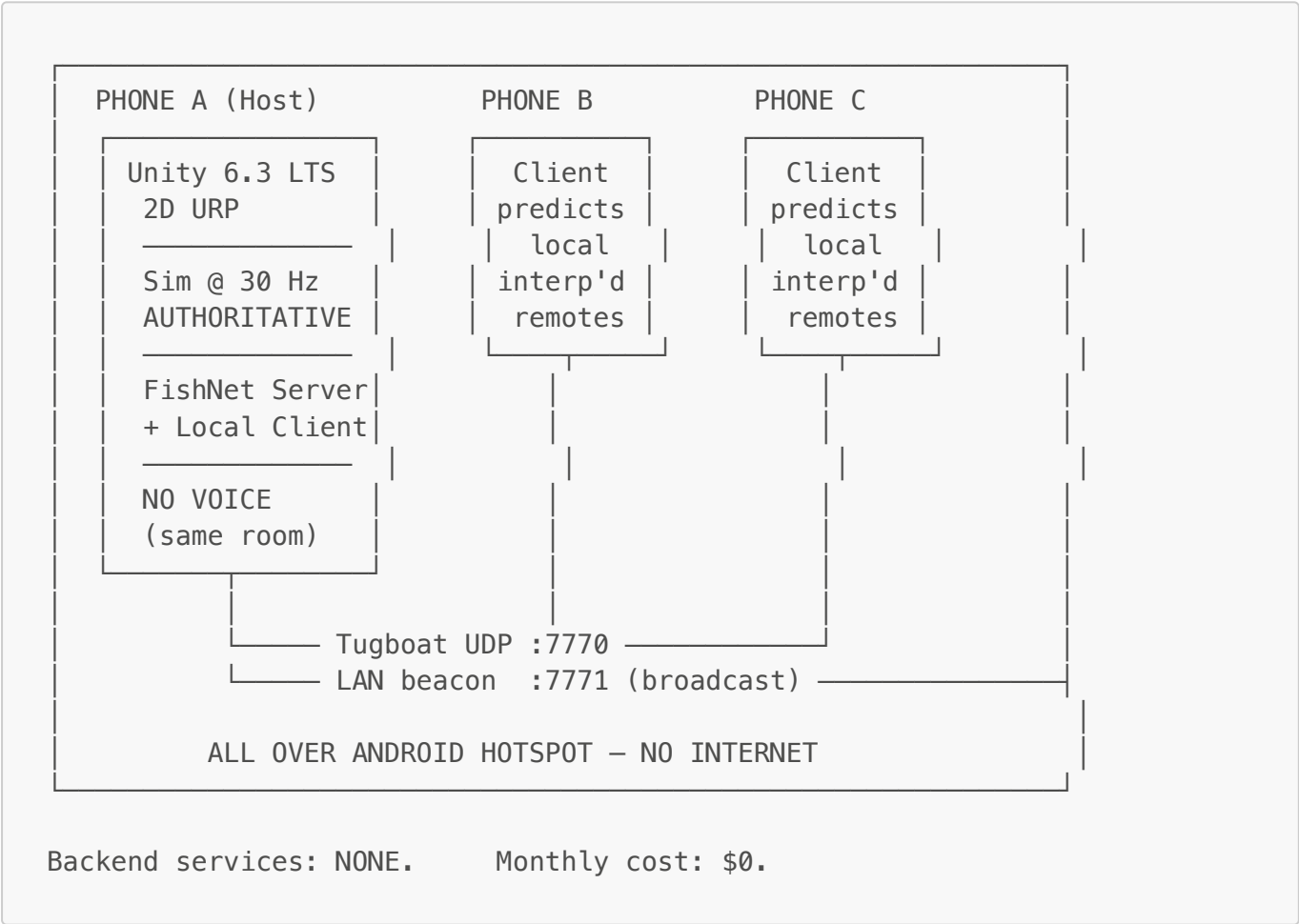
Reduce it: Ship LAN-only first, deliberately, at \$0/month. Hard-gate at Phase 1: **if it isn't fun single-player, stop and fix that before touching networking**. Write the "do not build" list from §7 somewhere you see it weekly. Put a playable build in a friend's hands every 2–3 weeks — external feedback is the only reliable motivation engine for solo work. Timebox art: placeholder rectangles until Phase 5.

When: Continuously. This is the one to actively manage.

Honorable mentions: art/asset pipeline consuming more time than all the code combined (very common — plan for it); Google Play target-API-level deadlines forcing an unplanned Unity upgrade; and FishNet major-version breaking changes mid-project (pin your version, upgrade deliberately between phases).

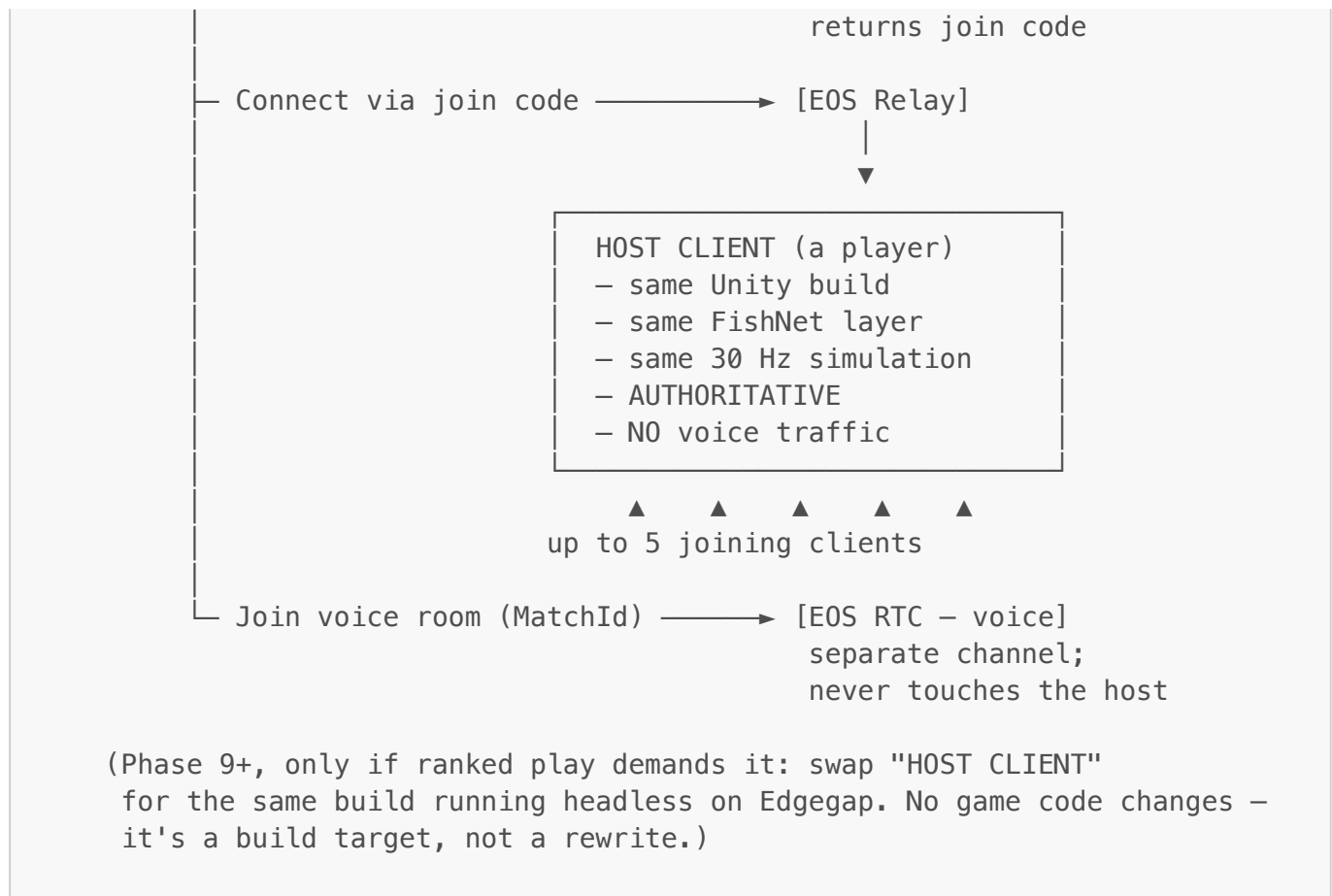
13. Recommended final architecture

v1.0 — LAN / hotspot (ship this first)



v2.0 — Online (adds a path, changes nothing above the transport)





The two things that make this architecture work

1. Voice is a separate channel, and LAN doesn't have it. Gameplay and voice have opposite requirements — one is tiny and latency-critical, the other is fat and latency-tolerant — so they run on separate paths. The host phone carries only the simulation, never audio. And because hotspot players are already sitting next to each other, LAN mode skips voice entirely: no mic permission in v1, no echo problem, no cost, two fewer weeks before launch.

2. Transport is the only swappable part. The simulation, replication, match state, lobby logic and prediction are byte-identical across LAN and online. Phase 7 is genuinely *additive* — you implement one interface and add a lobby screen. You are not writing the game twice.

14. Your first playable prototype

Goal: two phones on a hotspot, two colored squares shooting each other. Nothing else.

Not the lobby. Not voice. Not art. Not the map. **Two squares.**

This is deliberately the ugliest possible version, because it proves the two things that could actually kill the project — *does LAN discovery work on real Android devices*, and *does a networked jetpack feel good on a touchscreen* — while you've invested three weeks instead of three months.

Exact stack

Layer	Choice
Engine	Unity 6.3 LTS , 2D (URP) template
Language	C#
Networking	FishNet (free, Unity Asset Store or GitHub)
Transport	Tugboat (FishNet's default UDP/LiteNetLib), port 7770
Discovery	Hand-written System.Net.Sockets.UdpClient broadcast, port 7771
Android plugin	AndroidJavaObject wrapper for WifiManager.MulticastLock
Input	Unity Input System + On-Screen Stick components
Physics	Custom kinematic controller (Physics2D.BoxCast) — no Rigidbody2D on players
Art	Sprites-Default colored squares. Zero art assets.
Testing	ParrelSync (two Editor instances) + 2 physical Android phones
Build	IL2CPP, ARM64, Development Build ON (for the profiler)

Scope — 6 items, nothing more

1. One flat scene: a floor, two walls, a ceiling.
2. Player = colored square. Left stick moves; hold a button for jetpack thrust with a fuel bar.
3. Right stick aims a line and auto-fires a hitscan raycast every 0.2 s.
4. 100 HP, host-authoritative damage, death, respawn after 3 s at a random point.
5. Host button + Join button. Host broadcasts a beacon; Join lists rooms and connects by tapping.
6. On-screen debug text: **RTT · tick · reconciles/sec · packets in/out · kills**.

No lobby, menu, voice, scoreboard, match timer, weapons, grenades, sound, animation, or art.

Definition of done

Phone A creates a hotspot from Android settings. Phone B joins the hotspot. Both open the app. A taps Host. B taps Join, sees the room within 3 seconds, taps it, connects. Both squares fly around with jetpacks and shoot each other. Kills register. Movement feels responsive on **both** the host and the client.

With the router unplugged and mobile data off on both phones.

Recommended build order (~3 weeks)

Days	Work
1–3	Unity project, 2D URP, square that moves + jetpacks offline. Deploy to a real phone on day 3.
4–6	Touch controls: dual on-screen sticks, aim line, auto-fire raycast, hit flash. Tune the jetpack until it feels good <i>offline</i> .

Days	Work
7–12	Add FishNet. Host + client in ParrelSync over localhost. NetworkObject , input replication, prediction/reconcile, interpolated remotes, host-authoritative damage. Build the debug overlay.
13–16	Test host-on-phone + client-on-Editor over home Wi-Fi. Fix the first wave of prediction bugs (there will be a wave).
17–21	UDP beacon broadcaster + listener. MulticastLock plugin. Minimal host/join UI. Test on a real hotspot with the router off.

The gate

If at the end of this you find the jetpack movement *isn't fun* — stop and fix that before writing another line of networking. Everything downstream in this plan assumes the core loop is enjoyable. It's much cheaper to discover that in week 3 than in month 6.

Summary of the decisions to lock in today

Decision	Choice
Engine	Unity 6.3 LTS , 2D URP
Networking	FishNet (free) + Pro (~\$10) for lag comp later
Topology	Authoritative listen-server (a player hosts)
LAN transport	Tugboat UDP, direct IP
LAN discovery	UDP broadcast + QR + manual IP (all three)
Voice — LAN	None, by design — players are in the same room
Voice — online	EOS RTC (Phase 8) — free, separate channel, never touches the host
Voice default	Latching push-to-talk , 3-speaker cap
Tick rate	30 Hz sim / 20 Hz snapshots
Host migration	Don't build it. Fast re-host instead.
Backend for v1	None.
Online (Phase 7)	EOS (auth + lobby + relay), free — same networking layer
Ship order	LAN-only to Play Store first , online months later
v1 operating cost	\$0/month

Sources

- [Fish-Net: Features](#) · [Lag Compensation](#) · [Client-Side Prediction](#) · [Projectiles](#) · [Pro, Projects, and Support](#)

- [Dissonance — Choosing A Network](#) · [Dissonance Voice Chat \(Asset Store\)](#) · [Dissonance for FishNet integration](#) · [Voice chat in Unity and the problems therein](#)
- [Epic Online Services](#) · [EOS P2P Interface Reference](#) · [PlayEveryWare EOS Plugin for Unity](#)
- [Photon Fusion Pricing](#) · [Photon Pricing & CCU Plans](#)
- [Unity Gaming Services Billing FAQ](#) · [UGS Pricing Estimator](#) · [Unity is Canceling the Runtime Fee](#) · [Unity 6 Releases & Support](#)
- [NGO — Distributed authority topologies](#) · [NGO — Dealing with latency](#)
- [Android — Request permission to access nearby Wi-Fi devices](#) · [Behavior changes: Android 13+](#) · [Wi-Fi Direct service discovery](#)
- [Edgegap — Unity game server hosting](#) · [Godot 4 Multiplayer: Best Practices & Benchmarks \(2026\)](#) · [Mini Militia \(Wikipedia\)](#)